

파이썬 프로그래밍 3일차 강의자료

함수 · 모듈과 표준 라이브러리 · 문자열과 정규표현식

오늘의 한 줄 목표

반복되는 코드를 함수로 묶고, 모듈을 가져오며, 문자열을 자유자재로 다룬다.

대상: 비전공자 1~4학년 · 구성: 이론 + 실습

일정



오늘의 시간표

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

교시	주제	이론	실습
1	함수 기초	def·매개변수·return	계산기·BMI
2	함수 심화	기본값·가변인자·스코프	소수·텍스트 통계
3	모듈 & 표준 라이브러리	import·math·random	비밀번호·D-Day
4	문자열 다루기	split·join·replace·strip	문장 분석·전화번호
5	정규표현식 입문	re·패턴·findall	이메일·URL 검사

오늘의 전환

- 반복되는 코드를 함수로 묶습니다.
- 표준 라이브러리로 강력한 도구를 가져옵니다.
- 문자열과 정규식으로 실제 텍스트를 다룹니다.

TAKEAWAY

오늘의 목표: 코드를 함수로 나누고, 모듈을 가져오고, 텍스트를 다루는 기본기를 갖춘다.

복습

어제 복습 & 오늘의 연결고리

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

어제까지 만든 힘

- 변수·조건문·반복문·리스트·딕셔너리로 데이터를 처리했습니다.
- 여러 값을 저장하고 반복해서 계산하는 흐름을 익혔습니다.
- 하지만 코드가 길어질수록 같은 내용이 반복되기 시작합니다.

오늘의 해결

- 함수로 코드를 이름 붙여 재사용합니다.
- 모듈로 이미 만들어진 기능을 가져옵니다.
- 문자열과 정규식으로 텍스트를 자르고 검사합니다.

끝나면 만들 수 있는 것

- 랜덤 비밀번호 생성기
- D-Dav 계산기

TAKEAWAY

복불의 한계는 함수로, 직접 구현의 한계는 모듈로 넘습니다.

1교시

1교시 — 함수 기초

시간 구성: 이론 20분 → 실습 30분 → 휴식 10분 목표: 함수를 정의하고 호출하여 코드를 재사용 가능한 부품으로 만든다.

이론 20분

실습 30분

휴식 10분

이론

코드가 반복되고 있다

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 비슷한 코드가 계속 반복됨
- 매번 복붙 → 수정도 일일이 → 실수 발생
- 해결: 함수로 묶어 이름 붙이기!

Python example

```
print("=" * 20)
print("환영합니다")
print("=" * 20)
# ... 또 어딘가에서
print("=" * 20)
print("감사합니다")
print("=" * 20)
```

TAKEAWAY

비슷한 코드가 계속 반복됨

이론

함수(Function)란?

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

핵심 개념

- 함수 = 입력을 넣으면 결과를 주는 기계
- 비유: 자판기 — 돈(입력) 넣으면 음료(출력)

적용 질문

- 비유: 믹서기 — 과일(입력) 넣으면 주스(출력)
- 한 번 만들어두면 몇 번이고 재사용

TAKEAWAY

함수 = 입력을 넣으면 결과를 주는 기계

이론

사실 이미 함수를 쓰고 있었다

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 지금까지 쓴 print, len, int 가 전부 함수!
- 오늘은 이것 직접 만들어 봅니다

Python example

```
print("안녕")    # print: 출력 함수
len([1, 2, 3])  # len: 개수 세는 함수
int("123")      # int: 형변환 함수
input("이름: ") # input: 입력 함수
```

TAKEAWAY

지금까지 쓴 print, len, int 가 전부 함수!

함수 정의 — def

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- def + 함수이름 + () + 콜론 :
- 들여쓰기된 부분 = 함수의 내용(몸통)
- 아직 정의만 한 상태 (실행 x)

Python example

```
def greet():  
    print("안녕하세요!")  
    print("환영합니다!")
```

TAKEAWAY

def + 함수이름 + () + 콜론 :

이론

함수 호출 — 실행하기

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 함수이름() 으로 호출해야 실제로 실행
- 한 번 정의 → 여러 번 호출
- 출력:

Python example

```
def greet():  
    print("안녕하세요!")  
  
greet() # 함수 호출 → 실행됨!  
greet() # 또 호출 → 또 실행  
  
안녕하세요!  
안녕하세요!
```

TAKEAWAY

함수이름() 으로 호출해야 실제로 실행

정의 먼저, 호출은 나중

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 함수는 정의가 먼저, 호출은 그 뒤에
- 컴퓨터는 위에서 아래로 읽으니까
- 보통 함수들을 파일 위쪽에 모아 정의

Python example

```
greet()      # 에러! 아직 정의 안 됨

def greet():
    print("안녕")
```

TAKEAWAY

함수는 정의가 먼저, 호출은 그 뒤에

매개변수 — 입력 받기

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 괄호 안 name = 매개변수(parameter)
- 호출할 때 값을 넣으면 → 함수 안에서 그 값 사용
- 같은 함수, 다른 입력 → 다른 결과

Python example

```
def greet(name):  
    print(f"{name}님, 안녕하세요!")  
  
greet("홍길동") # 홍길동님, 안녕하세요!  
greet("김철수") # 김철수님, 안녕하세요!
```

TAKEAWAY

괄호 안 name = 매개변수(parameter)

이론

매개변수 vs 인자

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 매개변수: 함수 정의의 괄호 안 변수 (받을 그릇)
- 인자: 호출할 때 실제로 넣는 값
- 용어는 헛갈려도 OK, "넣는 값 = 인자" 정도로

Python example

```
def add(a, b):    # a, b = 매개변수(parameter)
    print(a + b)

add(3, 5)        # 3, 5 = 인자(argument)
```

TAKEAWAY

매개변수: 함수 정의의 괄호 안 변수 (받을 그릇)

이론

매개변수 여러 개

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 콤마로 매개변수 여러 개 가능
- 순서대로 들어감: name←"홍길동", age←24, city←"서울"
- 개수가 안 맞으면 에러

Python example

```
def introduce(name, age, city):  
    print(f"{name}, {age}살, {city} 거주")  
  
introduce("홍길동", 24, "서울")
```

TAKEAWAY

콤마로 매개변수 여러 개 가능

반환값 — return

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- return = 함수의 결과를 호출한 곳으로 돌려줌
- 돌려받은 값을 변수에 저장해 활용
- print 는 화면 출력, return 은 값 전달 (다름!)

Python example

```
def add(a, b):  
    return a + b    # 결과를 돌려줌  
  
result = add(3, 5)  
print(result)      # 8
```

TAKEAWAY

return = 함수의 결과를 호출한 곳으로 돌려줌

이론

return vs print 차이 ★

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- print 는 보여주기만, return 은 값을 남김

Python example

```
def add1(a, b):  
    print(a + b)    # 화면에 보여줄 뿐  
def add2(a, b):  
    return a + b    # 값을 돌려줌  
  
x = add1(3, 5)    # 8 출력, 하지만 x는 None!  
y = add2(3, 5)    # y = 8 (재사용 가능)  
print(y * 10)    # 80
```

TAKEAWAY

print 는 보여주기만, return 은 값을 남김

이론

return이 없으면?

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- return 이 없는 함수는 None(아무 값 없음)을 돌려줌
- 출력만 하는 함수는 return이 없어도 됨
- 계산 결과가 필요하면 반드시 return

Python example

```
def greet(name):  
    print(f"{name}님 안녕")  
  
result = greet("홍길동")  
print(result)  # None
```

TAKEAWAY

return 이 없는 함수는 None(아무 값 없음)을 돌려줌

이론

return은 함수를 끝낸다

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- return 을 만나면 함수가 즉시 종료
- 조건별로 다른 값을 돌려줄 수 있음

Python example

```
def check(score):  
    if score >= 60:  
        return "합격"  
    return "불합격" # else 없이도 OK  
  
print(check(80)) # 합격  
print(check(50)) # 불합격
```

TAKEAWAY

return 을 만나면 함수가 즉시 종료

좋은 함수 이름 짓기

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 변수처럼 함수도 이름이 곧 설명
- 관례: 동사로 시작(`calculate_`, `get_`, `make_`), 참/거짓은 `is_`
- `snake_case` (소문자 + 밑줄)

Python example

```
def f(x): ...      # 나쁜 예 (무슨 함수?)
def calculate_bmi(w, h): ... # 좋은 예 (읽으면 알)
def is_adult(age): ... # 참/거짓 반환은 is_로 시작
def get_average(nums): ... # 값을 가져오면 get_
```

TAKEAWAY

변수처럼 함수도 이름이 곧 설명

정리

함수를 쓰는 이유 정리

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

오늘의 결과물

- 재사용: 한 번 만들어 여러 번
- 수정 편함: 한 곳만 고치면 전부 반영
- 읽기 쉬움: `calculate_bmi()` 처럼 이름이 설명
- 분업: 큰 문제를 작은 함수로 나눠 해결
- "비슷한 코드가 2번 이상" → 함수로!

다음 연결

- 파일 읽기/쓰기와 예외처리로 프로그램을 더 단단하게 만듭니다.
- 함수로 나눈 코드를 작은 프로젝트 구조로 확장합니다.
- 실제 데이터 파일을 다루는 흐름으로 넘어갑니다.

TAKEAWAY

핵심은 코드를 작게 나누고, 필요한 도구를 가져오고, 텍스트를 정확히 다루는 것입니다.

실습

사칙연산 함수 만들기

작게 만들고, 입력값을 바꾸고, 결과를 직접 설명하는 실습

1. 따라 치기

2. 값 바꾸기

3. 결과 설명

실습 코드

```
def add(a, b):  
    return a + b  
def sub(a, b):  
    return a - b  
def mul(a, b):  
    return a * b  
def div(a, b):  
    return a / b  
x = int(input("첫 숫자: "))  
y = int(input("둘째 숫자: "))  
# ... 계속
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 여러 경우를 확인합니다.

TAKEAWAY

작은 함수 하나를 제대로 만들면 같은 코드를 여러 번 다시 쓸 수 있습니다.

실습

BMI 계산 함수

작게 만들고, 입력값을 바꾸고, 결과를 직접 설명하는 실습

1. 따라 치기

2. 값 바꾸기

3. 결과 설명

실습 코드

```
def calculate_bmi(weight, height_cm):  
    height_m = height_cm / 100  
    return weight / (height_m ** 2)  
  
bmi1 = calculate_bmi(60, 165)  
bmi2 = calculate_bmi(75, 180)  
print(f"사람1 BMI: {bmi1:.1f}")  
print(f"사람2 BMI: {bmi2:.1f}")
```

실습 포인트

- 여러 사람의 BMI를 함수 하나로 반복 계산!

TAKEAWAY

작은 함수 하나를 제대로 만들면 같은 코드를 여러 번 다시 쓸 수 있습니다.

실습

함수 만들기 (택1 이상)

작게 만들고, 입력값을 바꾸고, 결과를 직접 설명하는 실습

1. 따라 치기

따라 하기

- 원 넓이 함수: 반지름 $\rightarrow$ 넓이($3.14 r * 2$) 반환
- 온도 변환 함수: 섭씨 $\rightarrow$ 화씨 반환

2. 값 바꾸기

3. 결과 설명

변형 미션

- 인사 함수: 이름·시간대 받아 "좋은 아침/저녁" 인사
- 할인가 함수: 정가·할인율 받아 최종가 반환

TAKEAWAY

작은 함수 하나를 제대로 만들면 같은 코드를 여러 번 다시 쓸 수 있습니다.

휴식

휴식 10분

파일을 저장하고 다음 개념으로 넘어갈 준비

10 MIN BREAK

다음 교시: 함수 심화 — 기본값, 여러 입력, 변수의 범위(스코프)!

작성 파일 저장

질문 메모

입력값 실험

다음 실습 준비

TAKEAWAY

긴 코드 실습 전에는 파일을 저장하고 함수 이름을 다시 확인합니다.

2교시

2교시 — 함수 심화

시간 구성: 이론 20분 → 실습 30분 → 휴식 10분 목표: 기본값·키워드·가변 인자와 변수 스코프를 이해한다.

이론 20분

실습 30분

휴식 10분

이론

기본값 매개변수

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 매개변수에 = 기본값 지정
- 인자를 안 넣으면 기본값 사용, 넣으면 그 값 사용

Python example

```
def greet(name, greeting="안녕하세요"):
    print(f"{name}님, {greeting}!")

greet("홍길동")          # 홍길동님, 안녕하세요!
greet("김철수", "반갑습니다") # 김철수님, 반갑습니다!
```

TAKEAWAY

매개변수에 = 기본값 지정

이론

기본값 활용 예

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 자주 쓰는 값을 기본으로 → 호출이 간편
- 기본값 매개변수는 뒤쪽에 배치

Python example

```
def make_coffee(shots=1, size="regular"):
    print(f"{size} 사이즈, 샷 {shots}개")

make_coffee()          # regular, 샷 1개
make_coffee(2)         # regular, 샷 2개
make_coffee(2, "large") # large, 샷 2개
```

TAKEAWAY

자주 쓰는 값을 기본으로 → 호출이 간편

키워드 인자

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 호출 시 매개변수=값 으로 직접 지정
- 순서를 신경 안 써도 됨 (이름으로 매칭)
- 인자가 많을 때 가독성 ↑

Python example

```
def introduce(name, age, city):  
    print(f"{name}, {age}, {city}")  
  
introduce(age=24, city="서울", name="홍길동")
```

TAKEAWAY

호출 시 매개변수=값 으로 직접 지정

이론

위치 인자 vs 키워드 인자

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 섞어 쓸 땐 위치 인자가 먼저

Python example

```
def order(item, count):  
    print(f"{item} {count}개")  
  
order("커피", 2)          # 위치 인자 (순서대로)  
order(item="커피", count=2) # 키워드 인자 (이름 지정)  
order("커피", count=2)    # 섞어쓰기 (위치 먼저!)
```

TAKEAWAY

섞어 쓸 땐 위치 인자가 먼저

이론

가변 인자 *args

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- *numbers = 개수가 정해지지 않은 인자들을 튜플로 모음
- 몇 개를 넣든 OK!

Python example

```
def add_all(*numbers):  
    total = 0  
    for n in numbers:  
        total += n  
    return total  
  
print(add_all(1, 2, 3))    # 6  
print(add_all(1, 2, 3, 4, 5)) # 15
```

TAKEAWAY

***numbers = 개수가 정해지지 않은 인자들을 튜플로 모음**

*args 활용

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 메뉴 개수가 매번 달라도 하나의 함수로 처리
- 어제 배운 enumerate 와 결합

Python example

```
def print_menu(*items):  
    print("=== 메뉴 ===")  
    for i, item in enumerate(items, 1):  
        print(f"{i}. {item}")  
  
print_menu("아메리카노", "라떼", "콜드브루")
```

TAKEAWAY

메뉴 개수가 매번 달라도 하나의 함수로 처리

이론

****kwargs (키워드 가변 인자)**

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- info = 키워드 인자들**을 딕셔너리로 모음
- name="홍길동" 같은 쌍을 여러 개

Python example

```
def make_profile(**info):  
    for key, value in info.items():  
        print(f"{key}: {value}")  
  
make_profile(name="홍길동", age=24, city="서울")
```

TAKEAWAY

info = 키워드 인자들을 딕셔너리로 모음**

정리

인자 정리

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

종류	표기	모이는 형태
위치 인자	a, b	순서대로
기본값	a=10	안 주면 기본값
키워드 인자	a=10(호출)	이름으로
*args	*nums	튜플
**kwargs	**info	딕셔너리

제출/마무리

- 핵심을 한 줄씩 확인합니다.

TAKEAWAY

핵심은 코드를 작게 나누고, 필요한 도구를 가져오고, 텍스트를 정확히 다루는 것입니다.

이론

변수의 범위(스코프)

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 함수 안에서 만든 변수는 함수 안에서만 유효
- 이것을 스코프(범위) 라고 함

Python example

```
def my_func():  
    x = 10    # 함수 안에서 만든 변수  
    print(x)  
  
my_func()    # 10  
print(x)     # 에러! 밖에서는 못 봄
```

TAKEAWAY

함수 안에서 만든 변수는 함수 안에서만 유효

이론

지역 변수 (local)

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 지역 변수 = 함수 안에서만 사는 변수
- 함수가 끝나면 사라짐
- 그래서 함수끼리 변수 이름이 겹쳐도 안전

Python example

```
def calc():  
    result = 100  # 지역 변수  
    return result  
  
calc()  
print(result)    # 에러
```

TAKEAWAY

지역 변수 = 함수 안에서만 사는 변수

이론

전역 변수 (global)

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 전역 변수 = 함수 밖에서 만든 변수
- 함수 안에서 읽기는 가능

Python example

```
count = 0      # 전역 변수 (함수 밖)

def show():
    print(count) # 읽기는 가능

show()         # 0
```

TAKEAWAY

전역 변수 = 함수 밖에서 만든 변수

이론

전역 변수 수정의 함정

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 함수 안에서 전역 변수를 수정하려면 global 필요
- 하지만 global 남용은 권장 안 함(버그 유발)
- 더 좋은 방법: return으로 값을 주고받기

Python example

```
count = 0
def increase():
    count = count + 1 # 에러!

# 해결: global 키워드
def increase():
    global count
    count += 1
```

TAKEAWAY

함수 안에서 전역 변수를 수정하려면 global 필요

이론

return으로 주고받기 (권장)

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 전역 변수를 직접 건드리지 않고 값으로 소통
- 함수가 독립적이라 안전하고 예측 가능

Python example

```
count = 0
def increase(n):
    return n + 1    # 값을 돌려줌

count = increase(count) # 0 → 1
count = increase(count) # 1 → 2
print(count)          # 2
```

TAKEAWAY

전역 변수를 직접 건드리지 않고 값으로 소통

이론

docstring — 함수 설명 달기

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 함수 첫 줄에 `""" 설명 """` = docstring
- 이 함수가 무슨 일을 하는지 기록
- 나중의 나·동료가 사용법을 빠르게 파악

Python example

```
def calculate_bmi(weight, height):  
    """몸무게(kg)와 키(cm)로 BMI를 계산해 반환한다."""  
    return weight / ((height / 100) ** 2)  
  
help(calculate_bmi) # 설명을 볼 수 있음
```

TAKEAWAY

함수 첫 줄에 `""" 설명 """` = docstring

함수로 코드 나누기 (설계)

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 큰 일을 작은 함수로 쪼개 조립
- 각 함수는 한 가지 일만 (읽기·수정 쉬움)

Python example

```
def get_input():  
    return int(input("점수: "))  
def grade(score):  
    return "합격" if score >= 60 else "불합격"  
def show(result):  
    print(f"결과: {result}")  
  
show(grade(get_input())) # 입력→판정→출력
```

TAKEAWAY

큰 일을 작은 함수로 쪼개 조립

실습

소수 판별 함수

작게 만들고, 입력값을 바꾸고, 결과를 직접 설명하는 실습

1. 따라 치기

2. 값 바꾸기

3. 결과 설명

실습 코드

```
def is_prime(n):  
    if n < 2:  
        return False  
    for i in range(2, n):  
        if n % i == 0:  
            return False # 나누어지면 소수 아님  
    return True  
  
for num in range(2, 20):  
    if is_prime(num):  
        print(num, end=" ") # 2 3 5 7 11 13 17 19
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 여러 경우를 확인합니다.

TAKEAWAY

작은 함수 하나를 제대로 만들면 같은 코드를 여러 번 다시 쓸 수 있습니다.

실습

텍스트 통계 함수

작게 만들고, 입력값을 바꾸고, 결과를 직접 설명하는 실습

1. 따라 치기

2. 값 바꾸기

3. 결과 설명

실습 코드

```
def text_stats(text):  
    word_count = len(text.split())  
    char_count = len(text.replace(" ", ""))  
    return word_count, char_count # 두 값 반환(튜플)  
  
words, chars = text_stats("파이썬은 정말 재미있는 언어입니다")  
print(f"단어 수: {words}")  
print(f"글자 수(공백제외): {chars}")
```

실습 포인트

- return a, b → 여러 값을 튜플로 반환 (어제 언패킹!)

TAKEAWAY

작은 함수 하나를 제대로 만들면 같은 코드를 여러 번 다시 쓸 수 있습니다.

실습

함수 심화 (택1 이상)

작게 만들고, 입력값을 바꾸고, 결과를 직접 설명하는 실습

1. 따라 치기

따라 하기

- 가변 인자 평균 함수: `average(*nums)` → 평균 반환
- 최댓값 함수: 숫자 여러 개 받아 가장 큰 값 (`max` 안 쓰고)

2. 값 바꾸기

3. 결과 설명

변형 미션

- 요금 함수: 나이·기본값(평일/주말) 받아 요금 반환
- 카운트 함수: 문장과 글자 받아 그 글자가 몇 번 나오는지

TAKEAWAY

작은 함수 하나를 제대로 만들면 같은 코드를 여러 번 다시 쓸 수 있습니다.

휴식

휴식 10분

파일을 저장하고 다음 개념으로 넘어갈 준비

10 MIN BREAK

다음 교시: 모듈 — 남이 만든 강력한 도구를 가져다 쓰자!

작성 파일 저장

질문 메모

입력값 실험

다음 실습 준비

TAKEAWAY

긴 코드 실습 전에는 파일을 저장하고 함수 이름을 다시 확인합니다.

3교시

3교시 — 모듈 & 표준 라이브러리

시간 구성: 이론 20분 → 실습 30분 → 휴식 10분 목표: import로 외부 코드를 가져와 활용하고, 직접 모듈을 만든다.

이론 20분

실습 30분

휴식 10분

이론

모든 걸 직접 만들 순 없다

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

핵심 개념

- 랜덤 숫자, 날짜 계산, 복잡한 수학...
- 이걸 다 직접 만들면 너무 비효율적!

적용 질문

- 똑똑한 사람들이 이미 만들어 둔 도구가 있습니다
- 그게 바로 모듈/라이브러리 → 가져다 쓰면 됨

TAKEAWAY

랜덤 숫자, 날짜 계산, 복잡한 수학...

이론

모듈(Module)이란?

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

핵심 개념

- 모듈 = 함수·변수들을 모아둔 코드 파일
- 비유: 공구 상자 — 필요한 도구를 꺼내 씀

적용 질문

- 파이썬엔 수천 개의 모듈이 존재
- 예: random(랜덤), math(수학), datetime(날짜)

TAKEAWAY

모듈 = 함수·변수들을 모아둔 코드 파일

이론

import — 모듈 가져오기

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- import 모듈이름 으로 가져옴 (보통 파일 맨 위)
- 모듈이름.함수() 형태로 사용
- random.randint = random 모듈의 randint 함수

Python example

```
import random

print(random.randint(1, 6)) # 1~6 랜덤 (주사위)
```

TAKEAWAY

import 모듈이름 으로 가져옴 (보통 파일 맨 위)

이론

모듈.함수 사용법

개념을 짧게 이해하고 바로 함수.모듈.문자열 코드로 확인

개념 포인트

- 모듈.기능 으로 그 안의 함수.값을 사용
- math.pi 처럼 값(변수) 도 들어있음

Python example

```
import math

print(math.sqrt(16)) # 4.0 (제곱근)
print(math.pi)      # 3.141592... (원주율)
print(math.ceil(4.2)) # 5 (올림)
print(math.floor(4.8)) # 4 (내림)
```

TAKEAWAY

모듈.기능 으로 그 안의 함수.값을 사용

이론

from ... import

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 특정 함수만 콕 집어 가져오기
- 그러면 math. 을 안 붙여도 됨
- 자주 쓰는 함수에 편리

Python example

```
from math import sqrt, pi

print(sqrt(16)) # math. 없이 바로!
print(pi)
```

TAKEAWAY

특정 함수만 콕 집어 가져오기

이론

as — 별칭 붙이기

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- as 로 모듈에 짧은 별명
- 긴 이름을 줄여 편하게
- 데이터 분석에서 pd, np 가 관례

Python example

```
import random as rd  
print(rd.randint(1, 10))
```

실전에서 자주 보는 예 (다음 주)

```
# import pandas as pd  
# import numpy as np
```

TAKEAWAY

as 로 모듈에 짧은 별명

이론

math 모듈 둘러보기

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 복잡한 수학을 한 줄로!

Python example

```
import math

math.sqrt(25) # 5.0 제곱근
math.pow(2, 10) # 1024 거듭제곱
math.ceil(3.2) # 4 올림
math.floor(3.8) # 3 내림
math.factorial(5) # 120 팩토리얼
math.gcd(12, 18) # 6 최대공약수
```

TAKEAWAY

복잡한 수학을 한 줄로!

이론

random 모듈

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 어제 가위바위보·로또에서 이미 사용!

Python example

```
import random

random.randint(1, 6)      # 1~6 정수 (주사위)
random.random()           # 0~1 실수
random.choice(["가위", "바위", "보"]) # 하나 뽑기
random.sample(range(1,46), 6) # 6개 중복없이 (로또!)
random.shuffle(my_list)   # 리스트 섞기
```

TAKEAWAY

어제 가위바위보·로또에서 이미 사용!

이론

random 활용 맛보기

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 핵심 개념을 예시로 먼저 이해합니다.
- 코드로 직접 확인하며 감을 잡습니다.

Python example

```
import random

# 추천
names = ["김철수", "이영희", "박민수"]
winner = random.choice(names)
print(f"당첨자: {winner}")

# 랜덤 비밀번호용 숫자
code = random.randint(1000, 9999)
print(f"인증번호: {code}")
```

TAKEAWAY

핵심 개념을 예시로 먼저 이해합니다.

이론

datetime 모듈 — 날짜와 시간

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 오늘 날짜·현재 시각을 가져옴

Python example

```
from datetime import date, datetime

today = date.today()
print(today)      # 2026-06-25
print(today.year)  # 2026
print(today.month) # 6

now = datetime.now()
print(now)         # 2026-06-25 14:30:00
```

TAKEAWAY

오늘 날짜·현재 시각을 가져옴

이론

datetime — 날짜 계산 (D-Day)

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 날짜끼리 빼면 차이(일수)가 나옴
- D-Day, 나이, 기념일 계산에 활용

Python example

```
from datetime import date

today = date.today()
exam = date(2026, 12, 25)
dday = (exam - today).days
print(f"D-{dday}") # 남은 일수
```

TAKEAWAY

날짜끼리 빼면 차이(일수)가 나옴

이론

os 모듈 (살짝 맛보기)

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- os = 운영체제(파일·폴더) 관련 도구
- 파일 자동 정리 등 업무 자동화에 핵심 (6/29 파일 입출력과 연결)

Python example

```
import os

print(os.getcwd())    # 현재 폴더 경로
print(os.listdir())   # 폴더 안 파일 목록
# os.mkdir("새폴더")   # 폴더 만들기
```

TAKEAWAY

os = 운영체제(파일·폴더) 관련 도구

이론

표준 라이브러리란?

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

핵심 개념

- 표준 라이브러리 = 파이썬에 기본 포함된 모듈 모음
- 설치 없이 import 만 하면 바로 사용!

적용 질문

- 우리가 본 math, random, datetime, os 가 전부 표준
- "건전지 포함(batteries included)" 철학

TAKEAWAY

표준 라이브러리 = 파이썬에 기본 포함된 모듈 모음

내 모듈 직접 만들기

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 내가 만든 .py 파일도 모듈! 다른 파일에서 import

Python example

```
# mytools.py (파일로 저장)
def add(a, b):
    return a + b
def greet(name):
    return f"{name}님 안녕"

# main.py (같은 폴더)
import mytools
print(mytools.add(3, 5))    # 8
print(mytools.greet("홍길동")) # 홍길동님 안녕
```

TAKEAWAY

내가 만든 .py 파일도 모듈! 다른 파일에서 import

이론

pip와 외부 패키지

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- pip = 외부 패키지 설치 도구
- 표준에 없는 기능은 pip로 설치해 사용
- 예: requests(웹), pandas(데이터) — 마지막 주에!

Python example

```
pip install requests    # 터미널에서 설치  
  
import requests         # 설치 후 사용
```

TAKEAWAY

pip = 외부 패키지 설치 도구

실습

랜덤 비밀번호 생성기

작게 만들고, 입력값을 바꾸고, 결과를 직접 설명하는 실습

1. 따라 치기

2. 값 바꾸기

3. 결과 설명

실습 코드

```
import random
import string

length = int(input("비밀번호 길이: "))
chars = string.ascii_letters + string.digits # 영문+숫자
password = ""
for _ in range(length):
    password += random.choice(chars)
print(f"생성된 비밀번호: {password}")
```

실습 포인트

- string.ascii_letters = 영문 대소문자 전부

TAKEAWAY

작은 함수 하나를 제대로 만들면 같은 코드를 여러 번 다시 쓸 수 있습니다.

실습

D-Day 계산기

작게 만들고, 입력값을 바꾸고, 결과를 직접 설명하는 실습

1. 따라 치기

2. 값 바꾸기

3. 결과 설명

실습 코드

```
from datetime import date
print("=== D-Day 계산기 ===")
y = int(input("목표 연도: "))
m = int(input("목표 월: "))
d = int(input("목표 일: "))
target = date(y, m, d)
today = date.today()
dday = (target - today).days
if dday > 0:
    print(f"D-{dday} (앞으로 {dday}일)")
# ... 계속
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 여러 경우를 확인합니다.

TAKEAWAY

작은 함수 하나를 제대로 만들면 같은 코드를 여러 번 다시 쓸 수 있습니다.

실습

모듈 활용 (택1 이상)

작게 만들고, 입력값을 바꾸고, 결과를 직접 설명하는 실습

1. 따라 치기

따라 하기

- 주사위 게임: random으로 두 주사위, 합이 큰 사람 승
- 랜덤 퀴즈: 랜덤 두 수의 곱셈 문제 내고 정답 확인

2. 값 바꾸기

3. 결과 설명

변형 미션

- 원 계산기: math.pi로 원 넓이·둘레
- 내 나이 일수: 생년월일 받아 태어난 지 며칠인지

TAKEAWAY

작은 함수 하나를 제대로 만들면 같은 코드를 여러 번 다시 쓸 수 있습니다.

휴식

휴식 10분

파일을 저장하고 다음 개념으로 넘어갈 준비

10 MIN BREAK

다음 교시: 문자열 다루기 — 텍스트를 자르고.바꾸고.
합치기!

작성 파일 저장

질문 메모

입력값 실험

다음 실습 준비

TAKEAWAY

긴 코드 실습 전에는 파일을 저장하고 함수 이름을 다시 확인합니다.

4교시

4교시 — 문자열 다루기

시간 구성: 이론 20분 → 실습 30분 → 휴식 10분 목표: 문자열 메서드로 텍스트를 자유롭게 가공한다.

이론 20분

실습 30분

휴식 10분

이론

문자열은 시퀀스

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 문자열도 순서가 있는 모음(시퀀스)
- for, len, in 전부 사용 가능

Python example

```
s = "Hello"
for ch in s:
    print(ch)      # H e l l o (한 글자씩)
print(len(s))     # 5
print("e" in s)    # True
```

TAKEAWAY

문자열도 순서가 있는 모음(시퀀스)

복습

문자열 연결과 반복 (복습)

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

어제까지 만든 힘

- 변수·조건문·반복문·리스트·딕셔너리로 데이터를 처리했습니다.
- 여러 값을 저장하고 반복해서 계산하는 흐름을 익혔습니다.
- 하지만 코드가 길어질수록 같은 내용이 반복되기 시작합니다.

오늘의 해결

- 함수로 코드를 이름 붙여 재사용합니다.
- 모듈로 이미 만들어진 기능을 가져옵니다.
- 문자열과 정규식으로 텍스트를 자르고 검사합니다.

끝나면 만들 수 있는 것

TAKEAWAY

복불의 한계는 함수로, 직접 구현의 한계는 모듈로 넘습니다.

대소문자 변환

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- .메서드() 형태 = 문자열이 가진 기능
- 검색·비교 전 대소문자 통일에 유용

Python example

```
s = "Hello World"
print(s.upper())    # HELLO WORLD
print(s.lower())    # hello world
print(s.title())    # Hello World (단어 첫글자)
print(s.capitalize()) # Hello world (문장 첫글자)
```

TAKEAWAY

.메서드() 형태 = 문자열이 가진 기능

이론

공백 제거 — strip

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 사용자 입력의 불필요한 공백·줄바꿈 제거에 필수

Python example

```
s = " 안녕하세요 "
```

```
print(s.strip())    # "안녕하세요" (양끝 공백 제거)
```

```
print(s.lstrip())   # 왼쪽만
```

```
print(s.rstrip())   # 오른쪽만
```



```
email = " test@test.com \n"
```

```
print(email.strip()) # 깔끔하게
```

TAKEAWAY

사용자 입력의 불필요한 공백·줄바꿈 제거에 필수

이론

바꾸기 — replace

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- replace(찾을것, 바꿀것) → 모두 교체
- 전화번호 하이픈 제거 등 실전 활용

Python example

```
s = "I like cats"
print(s.replace("cats", "dogs")) # I like dogs
print("010-1234-5678".replace("-", "")) # 01012345678

# 여러 번 바꾸기
print("aaa".replace("a", "b")) # bbb
```

TAKEAWAY

replace(찾을것, 바꿀것) → 모두 교체

이론

나누기 — split ★

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- split(기준) → 기준으로 잘라 리스트로
- 기준 생략 시 공백 기준 (단어 분리!)

Python example

```
s = "사과,바나나,포도"
print(s.split(",")) # ['사과', '바나나', '포도']

sentence = "나는 파이썬을 배운다"
print(sentence.split()) # ['나는', '파이썬을', '배운다']
```

TAKEAWAY

split(기준) → 기준으로 잘라 리스트로

이론

합치기 — join ★

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- "연결자".join(리스트) → 리스트를 문자열로
- split의 반대 동작!

Python example

```
words = ["사과", "바나나", "포도"]  
print(", ".join(words)) # 사과,바나나,포도  
print(" ".join(words)) # 사과 바나나 포도  
print("".join(words)) # 사과바나나포도
```

TAKEAWAY

"연결자".join(리스트) → 리스트를 문자열로

이론

찾기 — find, index

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- find = 위치 반환, 없으면 -1
- index = 위치 반환, 없으면 에러
- 부분 문자열 검색에 사용

Python example

```
s = "Hello World"
print(s.find("World")) # 6 (위치)
print(s.find("Python")) # -1 (없으면 -1)
print(s.index("o")) # 4 (첫 번째 o)
```

TAKEAWAY

find = 위치 반환, 없으면 -1

이론

세기 — count

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- count(찾을것) → 몇 번 나오는지
- 어제 딕셔너리 빈도수보다 간단(특정 글자만)

Python example

```
s = "banana"
print(s.count("a"))    # 3
print(s.count("an"))   # 2

text = "도 레 미 도 도"
print(text.count("도")) # 3
```

TAKEAWAY

count(찾을것) → 몇 번 나오는지

이론

시작·끝 확인 — startswith, endswith

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 파일 확장자, URL 형식 확인 등에 유용

Python example

```
file = "report.pdf"
print(file.endswith(".pdf")) # True
print(file.startswith("report")) # True

url = "https://google.com"
print(url.startswith("https")) # True (보안 확인)
```

TAKEAWAY

파일 확장자, URL 형식 확인 등에 유용

이론

검사 메서드 — isdigit, isalpha

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 입력값 유효성 검사에 활용
- "숫자만 입력하세요" 확인 등

Python example

```
print("12345".isdigit()) # True (전부 숫자?)  
print("abc".isalpha()) # True (전부 글자?)  
print("abc123".isalnum()) # True (글자+숫자?)  
print(" ".isspace()) # True (전부 공백?)
```

TAKEAWAY

입력값 유효성 검사에 활용

이론

이스케이프 문자

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- \ + 특수 의미: \n(줄바꿈), \t(탭)
- 출력 모양을 다듬을 때

Python example

```
print("줄1\n줄2")    # \n = 줄바꿈
print("이름\t나이")  # \t = 탭
print("그는 \"안녕\"") # \" = 따옴표
print("C:\\Users")   # \\ = 백슬래시
```

TAKEAWAY

\ + 특수 의미: \n(줄바꿈), \t(탭)

정리

문자열 메서드 정리

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

메서드	기능
upper/lower	대소문자
strip	공백 제거
replace	바꾸기
split/join	나누기/합치기
find/count	찾기/세기
startswith	시작 확인
isdigit	숫자인지

제출/마무리

- 문자열은 불변 → 메서드는 새 문자열 반환(원본 유지)

TAKEAWAY

핵심은 코드를 작게 나누고, 필요한 도구를 가져오고, 텍스트를 정확히 다루는 것입니다.

실습

문장 분석기

작게 만들고, 입력값을 바꾸고, 결과를 직접 설명하는 실습

1. 따라 치기

2. 값 바꾸기

3. 결과 설명

실습 코드

```
text = input("문장을 입력하세요: ")

print(f"전체 길이: {len(text)}자")
print(f"공백 제외: {len(text.replace(' ', ''))}자")
print(f"단어 수: {len(text.split())}개")
print(f"대문자로: {text.upper()}")
words = text.split()
print(f"단어 목록: {words}")
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 여러 경우를 확인합니다.

TAKEAWAY

작은 함수 하나를 제대로 만들면 같은 코드를 여러 번 다시 쓸 수 있습니다.

실습

전화번호 정리기

작게 만들고, 입력값을 바꾸고, 결과를 직접 설명하는 실습

1. 따라 치기

2. 값 바꾸기

3. 결과 설명

실습 코드

```
phone = input("전화번호 입력(예: 010-1234-5678): ")

clean = phone.replace("-", "").replace(" ", "")
print(f"숫자만: {clean}")
print(f"유효한가? {clean.isdigit() and len(clean) == 11}")
# 가운데 마스킹
masked = clean[:3] + "*****" + clean[7:]
print(f"마스킹: {masked}")
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 여러 경우를 확인합니다.

TAKEAWAY

작은 함수 하나를 제대로 만들면 같은 코드를 여러 번 다시 쓸 수 있습니다.

실습

문자열 가공 (택1 이상)

작게 만들고, 입력값을 바꾸고, 결과를 직접 설명하는 실습

1. 따라 치기

따라 하기

- 회문 검사: 입력이 거꾸로 읽어도 같은지 (`s == s[::-1]`)
- 이메일 아이디 추출: @ 앞부분만 (`split("@")`)

2. 값 바꾸기

3. 결과 설명

변형 미션

- 단어 첫 글자 대문자: 각 단어 `title()` 또는 직접
- 이름 이니셜: "홍길동" → "홍ㄱㄷ" 같은 변형

TAKEAWAY

작은 함수 하나를 제대로 만들면 같은 코드를 여러 번 다시 쓸 수 있습니다.

휴식

휴식 10분

파일을 저장하고 다음 개념으로 넘어갈 준비

10 MIN BREAK

다음 교시: 정규표현식 — 복잡한 패턴(이메일·전화번호)을 한 번에 검사!

작성 파일 저장

질문 메모

입력값 실험

다음 실습 준비

TAKEAWAY

긴 코드 실습 전에는 파일을 저장하고 함수 이름을 다시 확인합니다.

5교시

5교시 — 정규표현식 입문

시간 구성: 이론 20분 → 실습 30분 → 휴식 10분 목표: 정규표현식으로 패턴을 매칭하는 기초를 익힌다. (깊이보다 감각!)

이론 20분

실습 30분

휴식 10분

이론

이런 검사를 하려면?

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

핵심 개념

- "이메일 형식이 맞나?" → @ 와 . 이 적절히 있나?
- "전화번호가 010-xxxx-xxxx 인가?"
- "비밀번호에 숫자가 들어있나?"

적용 질문

- 문자열 메서드로 하나하나 검사하면 너무 복잡
- 해결: 정규표현식(Regular Expression)!

TAKEAWAY

"이메일 형식이 맞나?" → @ 와 . 이 적절히 있나?

이론

정규표현식이란?

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

핵심 개념

- 정규표현식 = 문자열의 패턴을 표현하는 규칙
- 예: "숫자 3개 - 숫자 4개 - 숫자 4개" = 전화번호 패턴

적용 질문

- 짧은 기호로 복잡한 패턴을 표현
- 처음엔 암호처럼 보이지만, 읽는 법만 익히면 OK!

TAKEAWAY

정규표현식 = 문자열의 패턴을 표현하는 규칙

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 정규식은 re 모듈로 사용
- r"..." = raw 문자열(역슬래시를 그대로)
- 일단 "이렇게 쓰는구나" 정도로!

Python example

```
import re

text = "내 번호는 010-1234-5678 입니다"
result = re.search(r"\d{3}-\d{4}-\d{4}", text)
print(result.group()) # 010-1234-5678
```

TAKEAWAY

정규식은 re 모듈로 사용

이론

re의 주요 함수

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- search(첫 매칭), findall(모두), sub(치환)

Python example

```
import re
text = "apple 100 banana 200"

re.search(r"\d+", text)    # 첫 매칭 (100)
re.findall(r"\d+", text)   # 모두 찾기 ['100', '200']
re.match(r"apple", text)   # 맨 앞부터 매칭
re.sub(r"\d+", "X", text)  # 치환 'apple X banana X'
```

TAKEAWAY

search(첫 매칭), findall(모두), sub(치환)

메타문자 개요

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

핵심 개념

- 정규식은 특별한 의미의 기호(메타문자)로 패턴 표현
- 자주 쓰는 것들:
- `\d` 숫자, `\w` 글자/숫자, `\s` 공백

적용 질문

- `+` `*` `?` 반복, `{n}` 개수
- `[]` 문자 집합, `^` `$` 시작/끝
- 다음 슬라이드들에서 하나씩!

TAKEAWAY

정규식은 특별한 의미의 기호(메타문자)로 패턴 표현

\d — 숫자

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- \d = 숫자 한 글자 (0~9)
- \d\d = 숫자 두 글자
- 점수·전화번호 등 숫자 패턴에

Python example

```
import re
re.findall(r"\d", "a1b2c3")    # ['1', '2', '3']
re.findall(r"\d\d", "12 34 5") # ['12', '34']
```

TAKEAWAY

\d = 숫자 한 글자 (0~9)

이론

\w 와 \s

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- \w = 글자/숫자/밑줄 (word character)
- \s = 공백/탭/줄바꿈 (space)
- 대문자는 반대: \d(숫자 아님), \s(공백 아님)

Python example

```
import re
re.findall(r"\w", "a1_b") # ['a','1','_','b'] (공백 제외)
re.findall(r"\s", "a b c") # [' ',' '] (공백만)
```

TAKEAWAY

\w = 글자/숫자/밑줄 (word character)

이론

반복 — +, *, ?

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- + = 1개 이상, * = 0개 이상, ? = 0 또는 1개
- \d+ = "숫자가 연달아"

Python example

```
import re
re.findall(r"\d+", "a12b345") # ['12', '345'] (1개 이상)
re.findall(r"\d*", "a12")     # 0개 이상
re.findall(r"colou?r", "color colour") # 둘 다 매칭
```

TAKEAWAY

+ = 1개 이상, * = 0개 이상, ? = 0 또는 1개

이론

개수 지정 — {n}

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- {n} = 정확히 n개, {n,m} = n~m개
- 전화번호·우편번호처럼 자릿수 고정 패턴에 딱!

Python example

```
import re
re.findall(r"\d{3}", "1234567")    # ['123', '456']
re.search(r"\d{3}-\d{4}", "010-1234") # 010-1234
re.findall(r"\d{2,4}", "1 12 1234") # 2~4개
```

TAKEAWAY

{n} = 정확히 n개, {n,m} = n~m개

이론

문자 집합 — []

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- [] 안의 글자 중 하나라도 매칭
- [0-9] 숫자, [a-z] 소문자, [가-힣] 한글

Python example

```
import re
re.findall(r"[aeiou]", "hello") # ['e', 'o'] (모음)
re.findall(r"[0-9]", "a1b2")    # ['1', '2']
re.findall(r"[가-힣]+", "abc 가나다") # ['가나다'] (한글)
```

TAKEAWAY

[] 안의 글자 중 하나라도 매칭

이론

시작·끝 — ^ 와 \$

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- ^ = 문자열 시작, \$ = 문자열 끝
- "정확히 이 형식인가" 검사할 때 앞뒤를 고정
- 유효성 검사의 핵심!

Python example

```
import re
re.match(r"^010", "010-1234") # 010으로 시작?
re.search(r"\.com$", "a@b.com") # .com으로 끝?
```

TAKEAWAY

^ = 문자열 시작, \$ = 문자열 끝

이론

findall — 모두 추출

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- findall = 패턴에 맞는 것 전부 리스트로
- 텍스트에서 이메일·URL·숫자 추출에 최고

Python example

```
import re
text = "문의: a@test.com, b@gmail.com"
emails = re.findall(r"[\w.]+@[ \w.]+", text)
print(emails) # ['a@test.com', 'b@gmail.com']
```

TAKEAWAY

findall = 패턴에 맞는 것 전부 리스트로

이론

sub — 찾아서 바꾸기

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- sub(패턴, 바꿀것, 텍스트) = 패턴을 찾아 치환
- 민감정보 마스킹, 형식 통일에 활용

Python example

```
import re
text = "비밀번호는 1234, 인증은 5678"
print(re.sub(r"\d+", "****", text))
# 비밀번호는 ****, 인증은 ****

phone = "010.1234.5678"
print(re.sub(r"[.\s]", "-", phone)) # 010-1234-5678
```

TAKEAWAY

sub(패턴, 바꿀것, 텍스트) = 패턴을 찾아 치환

실전 패턴 예시

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

개념 포인트

- 외울 필요 없음! 필요할 때 찾아 쓰기(검색·AI 활용)
- 읽고 "대충 무슨 패턴인지" 아는 게 목표

Python example

```
# 전화번호: 010-1234-5678
r"010-\d{4}-\d{4}"

# 이메일: id@domain.com
r"^[\\w.]+@[\\w.]+\\. [a-z]+$"

# 우편번호(5자리)
r"^\\d{5}$"
```

TAKEAWAY

외울 필요 없음! 필요할 때 찾아 쓰기(검색·AI 활용)

정리

정규표현식 정리

개념을 짧게 이해하고 바로 함수·모듈·문자열 코드로 확인

오늘의 결과물

- re 모듈: search(첫), findall(모두), sub(치환)
- 자주: \d(숫자) \w(글자) +(1개+) {n}(개수) [] (집합) ^\$(시작끝)
- 완벽히 외우지 말 것 — 패턴은 검색/AI로, 읽는 감각이 중요
- 강력하지만 남용 금지(간단한 건 문자열 메서드로)

다음 연결

- 파일 읽기/쓰기와 예외처리로 프로그램을 더 단단하게 만듭니다.
- 함수로 나눈 코드를 작은 프로젝트 구조로 확장합니다.
- 실제 데이터 파일을 다루는 흐름으로 넘어갑니다.

TAKEAWAY

핵심은 코드를 작게 나누고, 필요한 도구를 가져오고, 텍스트를 정확히 다루는 것입니다.

실습

이메일·전화번호 검사

작게 만들고, 입력값을 바꾸고, 결과를 직접 설명하는 실습

1. 따라 치기

2. 값 바꾸기

3. 결과 설명

실습 코드

```
import re
email = input("이메일: ")
phone = input("전화번호(010-1234-5678): ")
if re.match(r"^[\\w.]+@[\\w]+\\. [a-z]+$", email):
    print("올바른 이메일")
else:
    print("이메일 형식 오류")
if re.match(r"^010-\\d{4}-\\d{4}$", phone):
    print("올바른 전화번호")
else:
    print("전화번호 형식 오류")
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 여러 경우를 확인합니다.

TAKEAWAY

작은 함수 하나를 제대로 만들면 같은 코드를 여러 번 다시 쓸 수 있습니다.

실습

텍스트에서 URL/숫자 추출

작게 만들고, 입력값을 바꾸고, 결과를 직접 설명하는 실습

1. 따라 치기

2. 값 바꾸기

3. 결과 설명

실습 코드

```
import re

text = "사이트 https://google.com 와 http://naver.com 방문, 가격  
5000원, 3개"

urls = re.findall(r"https?:\/\/[\w.]+", text)
numbers = re.findall(r"\d+", text)

print(f"URL: {urls}")
print(f"숫자: {numbers}")
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 여러 경우를 확인합니다.

TAKEAWAY

작은 함수 하나를 제대로 만들면 같은 코드를 여러 번 다시 쓸 수 있습니다.

실습

정규식 (택1 이상)

작게 만들고, 입력값을 바꾸고, 결과를 직접 설명하는 실습

1. 따라 치기

따라 하기

- 비밀번호 강도: 8자 이상 + 숫자 포함 검사 (`re.search(r"\d", pw)`)
- 해시태그 추출: 글에서 #단어 모두 찾기 (`r"#\w+"`)

2. 값 바꾸기

3. 결과 설명

변형 미션

- 날짜 추출: "2026-06-25" 형식 찾기 (`r"\d{4}-\d{2}-\d{2}"`)
- 숫자만 합산: 문장 속 모든 숫자 `findall` 후 합

TAKEAWAY

작은 함수 하나를 제대로 만들면 같은 코드를 여러 번 다시 쓸 수 있습니다.

휴식

오프라인 정리 & 휴식

잠깐 쉬고, 온라인에서는 "현실 도구"를 함수·모듈로 만들어봅니다!

WRAP-UP

함수(def·매개변수·return) · 기본값/가변 인자 · 스코프

작성 파일 저장

질문 메모

입력값 실험

다음 실습 준비

TAKEAWAY

긴 코드 실습 전에는 파일을 저장하고 함수 이름을 다시 확인합니다.